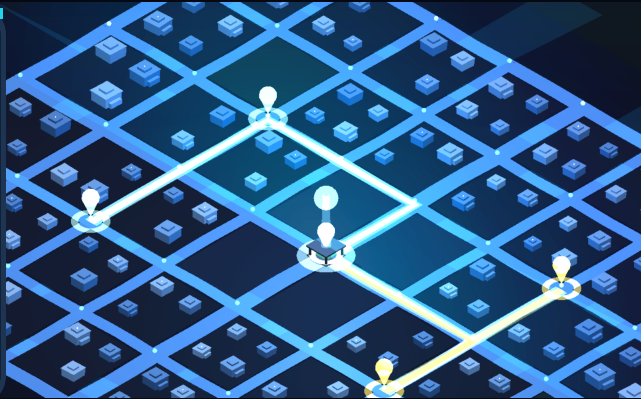


RoboRoute Nexus

A 3D last-mile control tower that turns a delivery scenario into an operable route plan, then explains every revision with local AI.

30-second pitch

RoboRoute Nexus is an offline-first logistics sandbox. Operators deploy robots, generate orders, run a bounded vehicle-routing optimisation, watch the plan move through a low-poly city, close a road, and ask a local copilot why the plan changed.



The problem

Last-mile operations are difficult to reason about when routes, capacity, closures and delays change together. The project makes the decision loop visible and repeatable instead of hiding it behind a black-box map.

The proof

The same local road graph powers the Three.js scene and the solver. A scenario revision is the shared unit between commands, KPIs, simulation events, copilot answers and shift reports.

Architecture in one sentence



The browser sends same-origin commands. The API owns state and revisions, the solver owns route quality, and the local model explains validated snapshots without mutating them.

Operator loop

- Configure: deploy 1-6 vehicles and generate 6-24 orders.
- Optimise: solve capacity, time-window, duration and cost constraints with OR-Tools.
- Operate: start, pause, speed up, relocate a robot, close a road and re-plan.
- Explain: ask Qwen about the current revision and export a grounded report.

KEY SIGNALS

Local-first

No paid AI API

Revision-safe

Atomic snapshots

Human-gated

AI never acts alone

Decisions worth explaining

The strongest interview answers are about boundaries: which component owns which truth, and why the system stays understandable under change.

Custom graph, not a map API

The fictional city is backed by a local graph with stable node and edge ids. Three.js renders the same dataset the router uses, so a barrier blocks a real edge rather than a visual region.

OR-Tools is the optimiser

The solver handles assignment, capacity, time windows and route duration. Economic cost is calculated separately, so internal objective units are never mislabeled as euros.

Revisions prevent stale UI

Every mutation carries a command id and scenario revision. The frontend keeps the highest accepted revision and drops late responses instead of showing an older plan over a newer one.

Docker keeps boundaries explicit

Frontend and API publish host ports. Ollama stays on the internal backend network, with a persistent model volume and cloud features disabled.

AI: interpreter, not optimiser

- The backend fixes qwen3:4b and the inference options; the browser cannot select arbitrary models.
- The model receives a validated scenario snapshot and a question, not an open-ended database connection.
- Responses are schema-validated, grounded in JSON field paths and tagged with the revision used.
- Proposals require human confirmation, while deterministic snapshot metrics anchor the report narrative.

Security and reliability talking points

Trust boundary

Browser -> API -> Ollama. The browser never sees Ollama and no model name is accepted from user input.

Failure behaviour

If Ollama is unavailable, the control tower still runs. The copilot reports a retryable error; routing and simulation remain useful.

Evidence

Playwright covers the landing, deck, live clock, road closures, responsive layout and the published docs page.

Deliberate limits

No auth, GPS, traffic, weather, mobile app, RAG, public Ollama or production-scale fleet claims.

Interview framing: every visual action is backed by a real state transition and a testable contract.

How to present it

Lead with the decision loop, then zoom into one tradeoff and one proof point. Keep the demo narrative shorter than the code tour.

A two-minute walkthrough

- 01 Open** The city starts idle. This proves startup has no hidden scenario generation or model download.
- 02 Plan** Deploy two robots, generate orders and run the bounded solver. Read distance, duration, cost and unassigned orders.
- 03 Operate** Start the clock at x4. Pause it. Close a road and show the coral edge, revision bump and detour.
- 04 Explain** Ask why the route changed. Show the grounded answer, human proposal gate and shift report download.

Elevator answer

I built an offline-first 3D last-mile control tower. The API owns a revisioned scenario, OR-Tools owns route optimisation, Three.js renders the same graph, and a local Qwen copilot explains validated snapshots without changing them autonomously. The result is a demo where every visual intervention has an operational consequence.

Questions you should be ready for

- Why not use Google Maps?** The fictional graph makes the demo offline, deterministic and visually coherent. It also lets a barrier block the exact edge used by the solver.
- Why local AI?** Privacy, reproducibility and no vendor dependency. The model explains computed facts; it is not trusted with optimisation or direct mutations.
- How do you avoid race conditions?** Command ids, scenario revisions, atomic snapshots and a frontend guard that rejects stale results.
- What would you build next?** Authentication, persisted run history, richer fleet constraints and production observability - after the current contract boundary is stable.

RUN IT

```
docker compose up --build
```

Open <http://localhost:8080> | API health: <http://localhost:8000/health>

Install Qwen only when the operator presses Install Qwen Core.